# Manual Test Plan — CLI Agent v0.5 + HOL Registry Agent + LLM Mode

**Pre-requisites:**

- `.env` configured with `HEDERA_TESTNET_ACCOUNT_ID` and `HEDERA_TESTNET_PRIVATE_KEY`
- `.env` configured with `ANTHROPIC_API_KEY` (for Parts E and F)
- HOL agent registered ( `deployments/hol-agent.json` exists)
- Agent account: `0.0.8218785` , Inbound topic: `0.0.8218788`

---

## Part A — CLI Agent: Annuity Lifecycle (Local Hardhat, Regex Mode)

Start the local stack: `./start.sh`

| # | Step | Command / Action | Expected Result |
|---|---|---|---|
| 1 | Health check | Type `health` | Output includes `API Health: OK` and `RPC Listening: true` |
| 2 | Help menu | Type `help` | Output shows four sections: **Lifecycle**, **Inspection**, **HCS-10 Agent Network**, **System** — plus a mode indicator at the bottom |
| 3 | Create deal | Type `create a deal with 3 coupons and face value 1000000` | Output shows a Correlation ID (e.g. `deal-17...`), an Annuity Contract address, a Stablecoin address, and Status: `created` |
| 4 | Execute deal | Type `execute the deal` | Output shows 3 coupon payment confirmations and final status `executed` |
| 5 | Transfer deal | Type `transfer the deal` | Output shows a secondary buyer address, a transfer price, and status `transferred` |
| 6 | Redeem deal | Type `redeem the deal` | Output shows redemption complete and status `redeemed` |
| 7 | List deals | Type `list deals` | The deal from step 3 appears with its correlation ID, current status, face value, and coupon count |
| 8 | Show transactions | Type `show transactions` | Tx log lists entries with hashes for create, execute, transfer, and redeem operations |

---

## Part B — HOL Registry Agent (Hedera Testnet)

| # | Step | Command / Action | Expected Result |
|---|---|---|---|
| 9 | Agent status | Run `node agent/hol-registry.js status` | Output shows account `0.0.8218785`, inbound topic `0.0.8218788`, outbound topic ID, |

| | | | |
|---|---|---|---|
| | | | profile topic ID, and 7 registered skills |
| 10 | Agent on-chain verification | Run `node agent/hol-registry.js status` and confirm the inbound topic exists on HashScan: `https://hashscan.io/testnet/topic/0.0.8218788` | HashScan shows the topic with messages (profile, skills). Note: the HOL Portal REST search does not index on-chain HCS-10 registrations — our agent won't appear there. |

---

## Part C — CLI Agent: HCS-10 Commands (Hedera Testnet, Regex Mode)

Start the CLI agent: `node agent/cli-agent.js --network hedera-testnet --no-llm`

| # | Step | Command / Action | Expected Result |
|---|---|---|---|
| 11 | Verify banner | Observe the startup banner | Banner shows `v0.5`, `HCS-10 Agent: 0.0.8218785`, `Inbound: 0.0.8218788`, and `Mode: regex` |
| 12 | List all agents | Type `list agents` | Output shows a table of agents from the HOL Registry (default query `agent`), each with name, registry source, and description |
| 13 | Search agents | Type `list agents imperium` | Output shows agents matching "imperium" (results from moltbook registry) with name, registry, and description. Note: our on-chain agent won't appear — the REST index doesn't crawl HCS-10 registrations. |
| 14 | Start listener | Type `listen` | Output shows `HCS-10 listener started` with 5-second polling interval |
| 15 | Stop listener | Type `stop listening` | Output shows `HCS-10 listener stopped` |

**Note:** The `connect to` and `show connections` commands are tested end-to-end in Part D, which requires a second terminal running the test requester agent to accept the connection.

---

## Part D — Agent-to-Agent Round-Trip (Hedera Testnet)

Open **two terminals** (T1 and T2).

| # | Step | Command / Action | Expected Result |
|---|---|---|---|
| 16 | Start CLI listener | **T1:** Run `node agent/cli-agent.js --network hedera-testnet --no-llm`, then type `listen` | T1 shows `HCS-10 listener started` |

| | | | |
|---|---|---|---|
| 17 | Run test requester | **T2:** Run `node agent/test-a2a.js` | T2 shows it is connecting to the Imperium agent and sending a skill request |
| 18 | Verify inbound reception | **T1:** Watch the listener output | T1 shows: (1) `Connection request from ...` auto-accepted, (2) `Skill request: annuity.status` received, (3) execution result displayed, (4) response sent confirmation |
| 19 | Verify requester response | **T2:** Watch the output | T2 shows the skill response received with status and result data |
| 20 | Send outbound skill | **T1:** Type `send annuity.status` | T1 sends the skill request and shows `Skill request sent. Polling for response...`. Note: the test requester has already exited, so no response will arrive — verify the message was submitted successfully, then Ctrl+C the poll. A full bidirectional test requires two CLI agent instances (future). |
| 21 | Show connections | **T1:** Type `show connections` | The connection from step 17 appears with its Connection Topic ID and remote agent account |

---

## Part E — LLM Mode: Natural Language Agent (Local Hardhat)

Start the local stack: `./start.sh` Start the CLI agent **without** `--no-llm` : `node agent/cli-agent.js`

**Pre-requisite:** `ANTHROPIC_API_KEY` must be set in `.env` .

| # | Step | Command / Action | Expected Result |
|---|---|---|---|
| 22 | Verify LLM banner | Observe the startup banner | Banner shows `v0.5`, `LLM-powered agent`, `Mode: LLM (Claude + hedera-agent-kit)` with a tool count (e.g. `21 tools`) |
| 23 | Natural language create | Type `I want to issue a new bond with 5 coupon payments and a face value of 2 million` | Claude understands the intent, calls the `create_annuity` tool, and returns a formatted response with Correlation ID, contract addresses, and status `created` |
| 24 | Freeform status check | Type `what's the status of my deal?` | Claude calls the `check_deal_status` tool (or asks for a deal ID if not clear). Returns deal status info. |
| 25 | Conversational balance query | Type `how much does each party have?` | Claude calls `show_balances` and returns formatted balance information for issuer, investor, secondary, and contract |
| 26 | Natural language execute | Type `settle and pay the coupons` | Claude calls `execute_deal` and returns execution results with transaction details |
| 27 | Freeform transfer | Type `sell the annuity for 800 thousand` | Claude calls `transfer_deal` with price 800000 and returns transfer confirmation |

| | | | |
|---|---|---|---|
| 28 | Natural language redeem | Type `redeem it now` | Claude calls `redeem_deal` and returns redemption confirmation |
| 29 | Help via natural language | Type `what can you do?` | Claude responds with a description of available capabilities without necessarily calling a tool |
| 30 | Ambiguous input | Type `show me everything about the last deal` | Claude intelligently selects one or more tools (status, balances, transactions) to provide a comprehensive answer |

---

## Part F — LLM Mode: Hedera-Native Queries (Hedera Testnet)

Start the CLI agent: `node agent/cli-agent.js --network hedera-testnet`

**Pre-requisite:** `ANTHROPIC_API_KEY` must be set in `.env`.

| # | Step | Command / Action | Expected Result |
|---|---|---|---|
| 31 | Verify LLM + Testnet | Observe the startup banner | Banner shows `v0.5`, `LLM-powered agent`, network `hedera-testnet`, and tool count including hedera-agent-kit tools |
| 32 | HBAR balance query | Type `what's my HBAR balance?` | Claude calls the `get_hbar_balance_query_tool` from hedera-agent-kit and returns the agent's HBAR balance |
| 33 | Account info lookup | Type `look up account 0.0.8199239` | Claude calls `get_account_query_tool` and returns Mirror Node account details (key, balance, memo, etc.) |
| 34 | Token balances | Type `what tokens does my account hold?` | Claude calls `get_account_token_balances_query_tool` and returns any HTS token associations |
| 35 | Topic info query | Type `show info for topic 0.0.8218788` | Claude calls `get_topic_info_query_tool` and returns topic details (admin key, submit key, memo) |
| 36 | Regex fallback | Exit, restart with `node agent/cli-agent.js --network hedera-testnet --no-llm` | Banner shows `Mode: regex`. Type `what's my HBAR balance?` — should show `I didn't understand that` (Hedera-native queries only work in LLM mode) |

---

## Part G — LLM Mode: HCS-10 via Natural Language (Hedera Testnet)

Continue from Part F in LLM mode (without `--no-llm`).

| # | Step | Command / Action | Expected Result |
|---|---|---|---|
| 37 | Discover agents naturally | Type `find me some agents on the network` | Claude calls `list_agents` and returns a formatted list of agents from the HOL Registry |

| | | | |
|---|---|---|---|
| 38 | Search agents naturally | Type `are there any agents related to imperium?` | Claude calls `list_agents` with query "imperium" and returns matching results |
| 39 | Start listener naturally | Type `start listening for incoming requests` | Claude calls `start_listener` and confirms the listener has started |
| 40 | Stop listener naturally | Type `stop the listener` | Claude calls `stop_listener` and confirms it stopped |

---

## Quick Smoke (6 tests, < 10 min)

1. **Step 1** — Health check (local API works)
2. **Step 3** — Create deal (core lifecycle works, regex mode)
3. **Step 9** — Agent status (HOL registration intact)
4. **Step 12** — List agents (HCS-10 discovery works)
5. **Step 22** — LLM banner (LLM mode initializes with tools)
6. **Step 23** — Natural language create (LLM understands domain intent)